# Rejoignez la séance

Scannez le QR code ou entrez le code sur votre téléphone.

COURS

# Sécurité

Qui es-tu ? Qu'as-tu le droit de faire ?

# FraisDirect a besoin de comptes

- **Alice**, cliente : elle passe et suit ses commandes
- **Bruno**, préparateur : il voit les commandes à préparer
- **Chloé**, administratrice : elle gère le catalogue

Personne d'autre ne doit voir les commandes d'Alice.

# Les trois A

1. **Authentification** : qui es-tu ?
2. **Autorisation** : as-tu le droit de faire ça ?
3. **Accounting** (traçabilité) : qu'as-tu fait ?

# Au programme

1. Prouver qui on est : **mots de passe**
2. Protéger une API : **Spring Security**
3. Ne plus renvoyer son mot de passe : **tokens et JWT**
4. Se connecter ailleurs : **SSO, OAuth, OpenID Connect**
5. Les droits : **RBAC et ABAC**
6. Garder une trace : **accounting**

ACTE 1

# Qui es-tu ?

Prouver son identité

# Trois façons de le prouver

| Ce que l'on… | Exemples |
| --- | --- |
| **sait** | un mot de passe, un code PIN |
| **possède** | un téléphone, une carte bancaire, une clé USB de sécurité |
| **est** | une empreinte digitale, un visage |

QUESTION

**Un code reçu par SMS prouve quelque chose que l'utilisateur…**

A. sait

B. possède

C. est

# Le mot de passe : comment l'envoyer ?

- **Basic Auth** : dans un header, à chaque requête
- **Digest Auth** : une variante de Basic, à éviter
- **Formulaire** : une requête dédiée à la connexion

# Basic Auth, étape par étape



# Construire le header

1. Identifiant et mot de passe : `alice:tomate42`
2. Encodés en Base64 : `YWxpY2U6dG9tYXRlNDI=`
3. Dans le header : `Authorization: Basic YWxpY2U6dG9tYXRlNDI=`

QUESTION

# Avec Basic Auth, le mot de passe envoyé est-il chiffré ?

# Base64 n'est pas du chiffrement

```
$ echo 'YWxpY2U6dG9tYXRlNDI=' | base64 -d
alice:tomate42
```

Basic Auth ne s'utilise **qu'en HTTPS** : c'est HTTPS qui chiffre la requête.

# Digest Auth : à éviter

- Le mot de passe n'est pas envoyé : le client envoie un hash calculé avec un nombre fourni par le serveur
- Pour vérifier ce hash, le serveur doit connaître le mot de passe : il ne peut pas le stocker de façon sûre
- Repose sur MD5, un algorithme cassé

Aujourd'hui : Basic Auth ou formulaire, **en HTTPS**.

QUESTION

## Comment FraisDirect doit-il stocker le mot de passe d'Alice en base ?

**A.** En clair

**B.** Chiffré avec une clé (AES)

**C.** Hashé avec SHA-256

**D.** Hashé avec BCrypt

# Stocker un mot de passe (1/3)

Jamais en clair, ni chiffré

- Les bases de données fuient : LinkedIn, Adobe, et bien d'autres
- Les gens réutilisent leurs mots de passe ailleurs
- Chiffré : celui qui vole la clé déchiffre tout

On stocke un **hash** : une empreinte qu'on ne peut pas inverser.

# Stocker un mot de passe (2/3)

Un hash seul ne suffit pas

- Deux personnes avec le même mot de passe ont le même hash
- Des tables géantes donnent le mot de passe d'un hash connu

On ajoute un **sel** : une valeur aléatoire, différente pour chaque utilisateur, mélangée au mot de passe.

# Stocker un mot de passe (3/3)

Un algorithme **lent**, fait pour ça : BCrypt

```
$2b$10$jpQqRmgJM0Er6Os2yTMFWu3lHfgjD7mp1xclQqTtadOgQOoSqUzcK
```

| Partie | Rôle |
| --- | --- |
| $2b$ | l'algorithme |
| 10 | le coût : $2^{10}$ tours |
| jpQqRmgJM0Er6Os2yTMFWu | le sel |
| 3lHfgjD7mp1x… | le hash |

# Sans mot de passe : le certificat client

- Le client présente un certificat, comme un serveur HTTPS
- ⊕ Rien à retenir, rien à taper
- ⊖ Il faut le renouveler avant son expiration
- ⊖ Le distribuer et le révoquer est compliqué

Exemple : l'accès à l'API Kubernetes

ACTE 2

# Spring Security

Protéger l'API FraisDirect

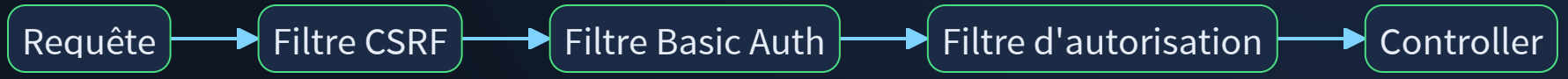
# Étape 1 : la dépendance

```
implementation 'org.springframework.boot:spring-boot-starter-security'
```

… et c'est tout : l'API est déjà protégée.

- Toutes les URL demandent une connexion
- Un utilisateur `user`, avec un mot de passe affiché dans les logs
- Un formulaire de connexion et Basic Auth

# Comment ça marche : des filtres




1. La requête traverse les filtres l'un après l'autre
2. Un filtre peut identifier l'utilisateur et le ranger dans le `SecurityContext`
3. Un filtre peut aussi arrêter la requête : 401 ou 403
4. Si tout passe, le controller est appelé

# Un filtre

```java
public class LogFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
            HttpServletRequest request,
            HttpServletResponse response,
            FilterChain chain)
            throws ServletException, IOException {
        log.info("Requête sur {}", request.getRequestURI());
        chain.doFilter(request, response);
    }
}
```

# Étape 2 : configurer la chaîne

```java
@Bean
SecurityFilterChain filterChain(HttpSecurity http)
        throws Exception {
    http
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated())
        .httpBasic(Customizer.withDefaults());
    return http.build();
}
```

# Étape 3 : pas de session en mémoire

- Par défaut, Spring garde l'utilisateur connecté dans une session HTTP
- Cette session vit dans la mémoire de l'instance
- Avec plusieurs instances derrière un load balancer : le panier perdu du cours middleware !

```
.sessionManagement(session -> session
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

# Étape 4 : ouvrir une partie de l'API

```
http
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/produits/**").permitAll()
        .requestMatchers("/commandes/**").authenticated()
        .anyRequest().denyAll())
    .httpBasic(Customizer.withDefaults());
```

Les règles sont lues **dans l'ordre** : la première qui correspond gagne.

QUESTION

## À vous : le catalogue est public, tout le reste demande une connexion

Le catalogue est servi sous `/produits`.

# Étape 5 : brancher nos utilisateurs

```java
@Service
public class ClientDetailsService
        implements UserDetailsService {
    // ...

    @Override
    public UserDetails loadUserByUsername(String email) {
        Client client = clientRepository.findByEmail(email)
                .orElseThrow(() ->
                        new UsernameNotFoundException(email));
        return User.withUsername(client.getEmail())
                .password(client.getMotDePasseHash())
                .roles(client.getRole())
                .build();
    }
}
```

# Étape 6 : comparer avec BCrypt

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

À l'inscription : `passwordEncoder.encode(motDePasse)`

À la connexion : Spring appelle `matches` tout seul

# Côté Angular : protéger les pages

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService);
  const router = inject(Router);
  return auth.estConnecte()
    ? true
    : router.parseUrl('/connexion');
};
```

```
{
  path: 'commandes',
  component: CommandesComponent,
  canActivate: [authGuard]
}
```

# Côté Angular : envoyer le header partout

```ts
export const authInterceptor: HttpInterceptorFn =
  (req, next) => {
    const token = inject(AuthService).token();
    if (!token) {
      return next(req);
    }
    return next(req.clone({
      setHeaders: { Authorization: `Bearer ${token}` }
    }));
  };
```

```ts
provideHttpClient(withInterceptors([authInterceptor]))
```

ACTE 3

# Les tokens

Ne plus envoyer son mot de passe à chaque requête

# Le problème de Basic Auth

- Le mot de passe voyage dans chaque requête
- Le front doit le garder en mémoire
- Chaque requête relance BCrypt, volontairement lent

# L'idée : échanger le mot de passe contre un token




Le mot de passe n'est envoyé qu'**une fois**.

# Token enregistré

1. À la connexion, l'API tire une valeur au hasard
2. Elle l'enregistre en base, avec l'utilisateur
3. À chaque requête, elle cherche le token en base

- ⊕ On peut le révoquer : il suffit de le supprimer
- ⊖ Une lecture en base à chaque requête

# Token signé

1. À la connexion, l'API écrit qui est l'utilisateur dans le token
2. Elle le signe avec un secret qu'elle seule connaît
3. À chaque requête, elle vérifie la signature : pas de base

- ⊕ Aucune base à interroger
- ⊖ Impossible à révoquer avant son expiration

# JWT : JSON Web Token

Le format de token signé le plus répandu : trois parties séparées par des points

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGljZSIsIm5hbWUiOiJBbGljZSIsInJvbGVzIjpbIkNMSUVOVCJdLCJpYXQiOjE3OTU0MzA0MDAsImV4cCI6MTc5NTQzNDAwMH0.kmSnlxdc6nSezfww2U7Qcse7mJTaiJNuLD0D9oJZLRY

1. Le header : l'algorithme de signature
2. Le payload : les informations sur l'utilisateur
3. La signature

# Ce qu'il contient

```
{ "alg": "HS256", "typ": "JWT" }
```

```
{
  "sub": "alice",
  "name": "Alice",
  "roles": ["CLIENT"],
  "iat": 1795430400,
  "exp": 1795434000
}
```

QUESTION

# Sans connaître le secret, peut-on lire le contenu d'un JWT ?

# Signé, pas chiffré

- Tout le monde peut **lire** un JWT
- Seul le détenteur du secret peut le **créer** ou le **modifier**

Jamais de mot de passe ni de donnée secrète dans un JWT.

# La signature

```
HMAC-SHA256( header + "." + payload , "secret-de-fraisdirect" )
```

Pour vérifier un token, l'API :

1. sépare les trois parties
2. recalcule la signature avec son secret
3. la compare avec celle du token
4. vérifie que `exp` n'est pas dépassé

QUESTION

**Alice décode son JWT, remplace `CLIENT` par `ADMIN` et le renvoie à l'API. Que se passe-t-il ?**

A. L'API l'accepte : Alice devient administratrice

B. L'API le rejette : la signature ne correspond plus

C. L'API lui redemande son mot de passe

ACTE 4

# Se connecter ailleurs

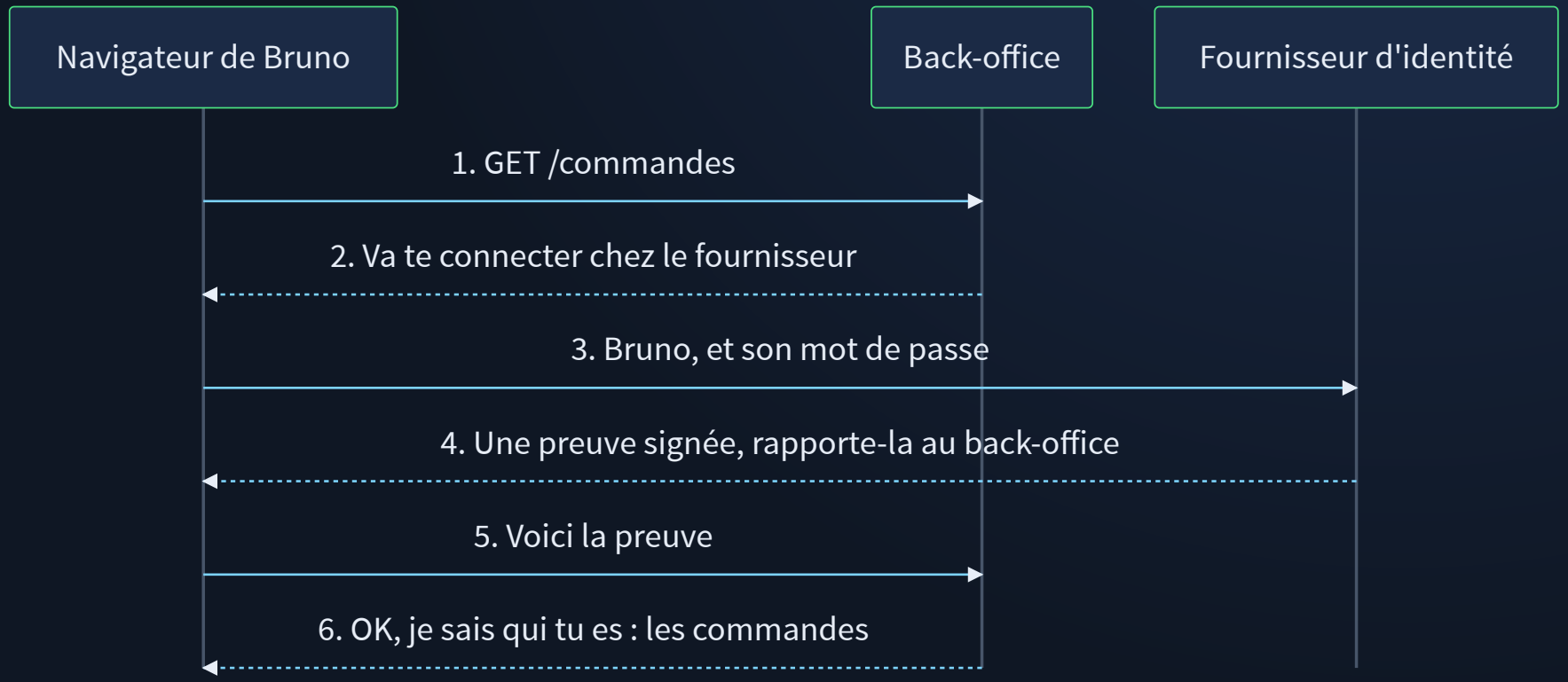
SSO, authentification forte, OAuth, OpenID Connect

# Le problème des employés

- Bruno utilise le back-office, l'outil de l'entrepôt, le planning…
- Un compte et un mot de passe par application
- Et le jour où il quitte FraisDirect, il faut les fermer un par un

Une seule connexion pour toutes les applications : le **Single Sign-On** (SSO).

# Le SSO avec SAML



# Ce que le SSO apporte

- Un seul mot de passe pour toutes les applications
- On ne se connecte qu'une fois
- On peut renforcer la connexion à un seul endroit
- Branché sur l'annuaire de l'entreprise (LDAP, Active Directory)

# L'authentification forte

Combiner au moins deux facteurs de l'acte 1

- Ce qu'on **sait** : le mot de passe
- + ce qu'on **possède** : une application d'authentification, une clé de sécurité, un SMS

  - ⊕ Un mot de passe volé ne suffit plus
  - ⊖ Une connexion un peu moins simple

# Un nouveau besoin

FraisDirect veut ajouter le créneau de livraison dans l'agenda Google d'Alice.

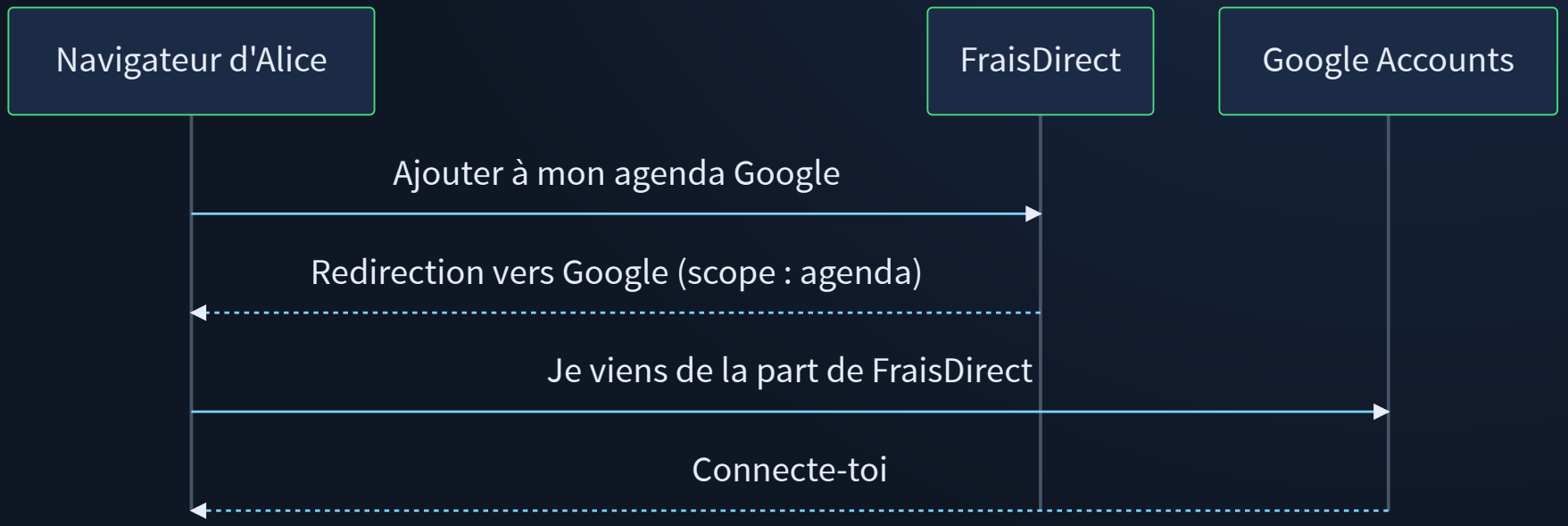
Mauvaise idée : demander à Alice son mot de passe Google.

Bonne idée : demander à Google une autorisation **limitée** à l'agenda : **OAuth 2**.

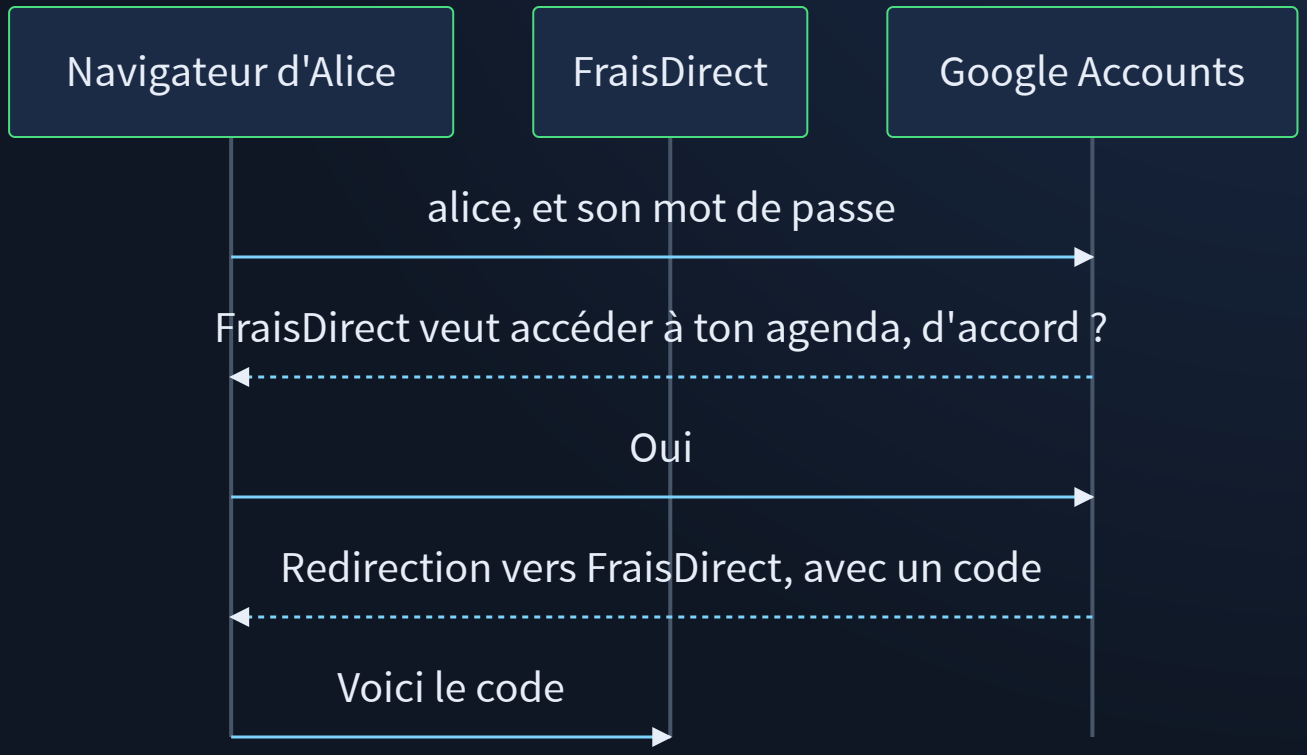
# Les acteurs d'OAuth

| Rôle | Dans l'exemple |
| --- | --- |
| Resource owner | Alice, propriétaire de son agenda |
| Client | FraisDirect, qui veut accéder à l'agenda |
| Authorization server | Google Accounts, qui délivre les tokens |
| Resource server | L'API Google Agenda |

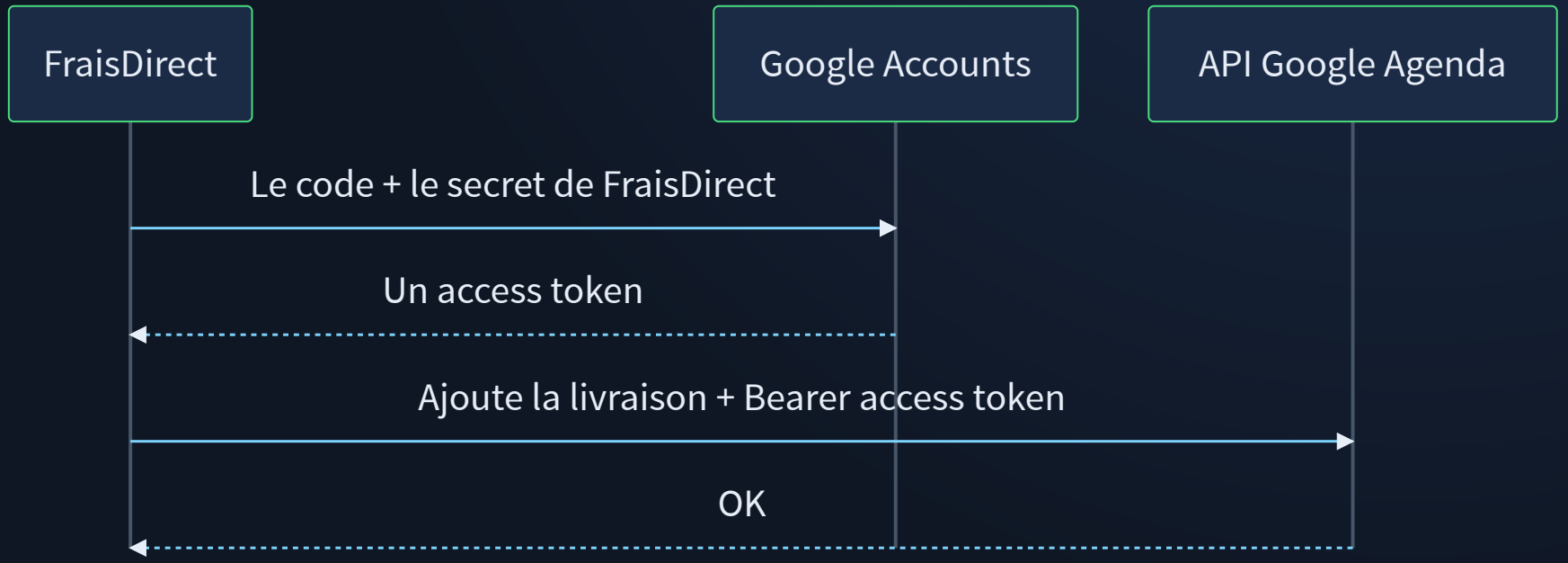
# OAuth étape par étape (1/3)



# OAuth étape par étape (2/3)



# OAuth étape par étape (3/3)



QUESTION

## Dans cet exemple, qui délivre l'access token ?

A. FraisDirect

B. Google Accounts

C. L'API Google Agenda

D. Le navigateur d'Alice

# OAuth autorise, il n'identifie pas

- L'access token dit : « ce porteur peut modifier cet agenda »
- Il ne dit pas à FraisDirect **qui** est Alice

Pour « Se connecter avec Google » : **OpenID Connect**.

# OpenID Connect (OIDC)

1. Les mêmes étapes qu'OAuth, avec le scope `openid`
2. En plus de l'access token, Google renvoie un **ID token**
3. L'ID token est un JWT qui décrit l'utilisateur


```
{
  "iss": "https://accounts.google.com",
  "sub": "110248495921238986420",
  "aud": "fraisdirect",
  "email": "alice@example.com",
  "name": "Alice",
  "exp": 1795434000
}
```

# La signature, sans secret partagé

- Google signe ses tokens avec une clé **privée** (algorithme RS256)
- Il publie la clé **publique** correspondante
- N'importe quelle API peut vérifier le token, sans pouvoir en fabriquer

# La découverte automatique

`https://accounts.google.com/.well-known/openid-configuration`


```
{
  "issuer": "https://accounts.google.com",
  "authorization_endpoint": "https://accounts.google.com/o/oauth2/v2/auth",
  "token_endpoint": "https://oauth2.googleapis.com/token",
  "jwks_uri": "https://www.googleapis.com/oauth2/v3/certs"
}
```

# Dans Spring : vérifier les tokens

```
implementation 'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
```

```
spring.security.oauth2.resourceserver.jwt.issuer-uri=https://accounts.google.com
```

```
http.oauth2ResourceServer(oauth2 -> oauth2
        .jwt(Customizer.withDefaults()));
```

1. Au démarrage, Spring lit la découverte de l'issuer
2. Il récupère les clés publiques
3. À chaque requête, il vérifie la signature et l'expiration du token

ACTE 5

# As-tu le droit ?

L'autorisation

QUESTION

**Bruno est bien connecté, mais il n'a pas le droit de modifier le catalogue. Quel code HTTP l'API doit-elle renvoyer ?**

A. 401 Unauthorized

B. 403 Forbidden

# 401 ou 403 ?

| Code | Signifie | Le client doit… |
|---|---|---|
| 401 | Je ne sais pas qui tu es | se connecter |
| 403 | Je sais qui tu es, et c'est non | abandonner |

# RBAC : des droits par rôle

| Rôle | Catalogue | Commandes |
|---|---|---|
| CLIENT (Alice) | lire | passer, suivre |
| PREPARATEUR (Bruno) | lire | voir, préparer |
| ADMIN (Chloé) | lire, modifier | tout |

# Les rôles dans Spring

1. `.roles("ADMIN")` dans le `UserDetailsService`
2. Spring en fait l'autorité `ROLE_ADMIN`
3. `hasRole("ADMIN")` cherche `ROLE_ADMIN`
4. `hasAuthority("ADMIN")` cherche exactement `ADMIN`

# Des règles par URL

```
1 http
2     .authorizeHttpRequests(auth -> auth
3         .requestMatchers(HttpMethod.GET, "/produits/**")
4             .permitAll()
5         .requestMatchers("/produits/**")
6             .hasRole("ADMIN")
7         .requestMatchers("/preparation/**")
8             .hasAnyRole("PREPARATEUR", "ADMIN")
9         .anyRequest().authenticated())
```

# Piège : mon POST renvoie 403

- Spring active la protection **CSRF** par défaut
- Elle protège les sessions par cookie
- Notre API est stateless, avec un header : pas de cookie à protéger

```
.csrf(csrf -> csrf.disable())
```

# Des règles sur les méthodes

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig { /* ... */ }
```

```
1 @PostMapping("/produits")
2 @PreAuthorize("hasRole('ADMIN')")
3 public Produit creer(@RequestBody Produit produit) {
4     return produitService.creer(produit);
5 }
```

# Une hiérarchie de rôles

```
@Bean
static RoleHierarchy roleHierarchy() {
    return RoleHierarchyImpl.fromHierarchy("""
            ROLE_ADMIN > ROLE_PREPARATEUR
            ROLE_PREPARATEUR > ROLE_CLIENT
            """);
}
```

Chloé a automatiquement les droits de Bruno, et Bruno ceux d'Alice.

# Le rôle ne suffit pas toujours

Alice et Bob sont tous les deux `CLIENT`.

Alice ne doit voir que **ses** commandes, pas celles de Bob.

La décision dépend d'un **attribut** : à qui appartient la commande.

# ABAC : des droits par attribut

- Le pays de l'utilisateur : le catalogue de Netflix
- L'âge : le contrôle parental de Disney+
- La relation : les publications « amis » de Facebook
- Le propriétaire : les commandes d'Alice

QUESTION

**« Un préparateur ne voit que les commandes de son entrepôt. » C'est une règle…**

A. RBAC : par rôle

B. ABAC : par attribut

# ABAC dans Spring

```
@GetMapping("/commandes/{id}")
@PostAuthorize("returnObject.client == authentication.name")
public Commande commande(@PathVariable Long id) {
    return commandeService.trouver(id);
}
```

1. La méthode s'exécute et charge la commande
2. Spring compare son client avec l'utilisateur connecté
3. S'ils diffèrent : 403

# RBAC et ABAC ensemble

- Les rôles donnent les grands droits
- Les attributs les affinent

ACTE 6

# Qu'as-tu fait ?

L'accounting

# Garder une trace

- **Qui** : Chloé
- **Quoi** : a changé le prix des tomates de 4,20 € à 0,42 €
- **Quand** : le 24 novembre à 9 h 12

Dans des logs dédiés, ou dans une table d'historique en base.

# À retenir

- Authentification, autorisation, accounting : dans cet ordre
- Un mot de passe se stocke hashé avec BCrypt, et voyage en HTTPS
- Un JWT est signé, pas chiffré
- OAuth autorise, OpenID Connect authentifie
- La sécurité se fait dans l'API, jamais seulement dans le front